B.TECH • PROJECT REVIEW 01

AI RESUME SCREENING SYSTEM

Smart screening. Faster shortlisting. Better hiring decisions.

FRONTEND + BACKEND + DATABASE

STUDENT NAME: B.Chennakesavulu | ROLL NO: 24691A2838

CLASS: III CST -A | REVIEW DATE: 18-May-2026 to 17-July-2026

INTERNSHIP CERTIFICATE

motherson

24th Aug 2026

TO WHOMSOEVER IT MAY CONCERN

This is to certify that **Chennakesavulu,** from **Madanapalle Institute of Technology & Science** has done his Internship Training at Motherson Automotive Technologies & Engineering, **Unit - II**, starting from 18th May 2026 to 17th July 2026.

During the Internship Period his conduct was good.

Wishing him every success.

For Motherson Automotive Technologies and Engineering
(A Division of Samvardhana Motherson International Limited)

Authorised Signatory

Project at a glance

01 Upload

Recruiter uploads PDF resumes and enters a job description.

02 Extract

Backend reads resume text and identifies relevant skills.

03 Match

Keyword-based matching calculates a similarity percentage.

04 Review

Dashboard displays candidate scores, profiles and reports.

Problem statement

Manual workload

Reviewing resumes individually takes significant time.

Inconsistent filtering

Different reviewers may interpret skill requirements differently.

Large applicant pools

Identifying relevant candidates becomes difficult at scale.

Slow shortlisting

Delays affect hiring and candidate communication.

Project objectives

Automate initial screening

Reduce repetitive manual resume review.

Match job requirements

Compare extracted resume skills with a job description.

Rank candidates

Display an interpretable match percentage.

Organize results

Provide profiles, search, dashboard and CSV export.

Abstract

The AI Resume Screening System is a web-based prototype designed to simplify the initial stage of recruitment. Recruiters enter a job title and description, then upload PDF resumes. A Python Flask backend extracts text from uploaded documents and compares relevant keywords with job requirements. The application presents candidate match percentages and makes results accessible through a dashboard, candidate list, detailed profiles, and reports.

The current implementation uses rule-based keyword matching rather than a trained machine-learning model. It is intended to demonstrate how resume processing, matching logic, a web interface and database storage can work together to support faster initial shortlisting.

PROJECT REVIEW 01

Existing system vs proposed system

EXISTING • Manual review

HR teams read each resume and compare it against the job description.

LIMITATION • Time & consistency

Repeated checks are slow and can vary between reviewers.

PROPOSED • Web-based screening

Upload multiple resumes, extract skills and compute match scores.

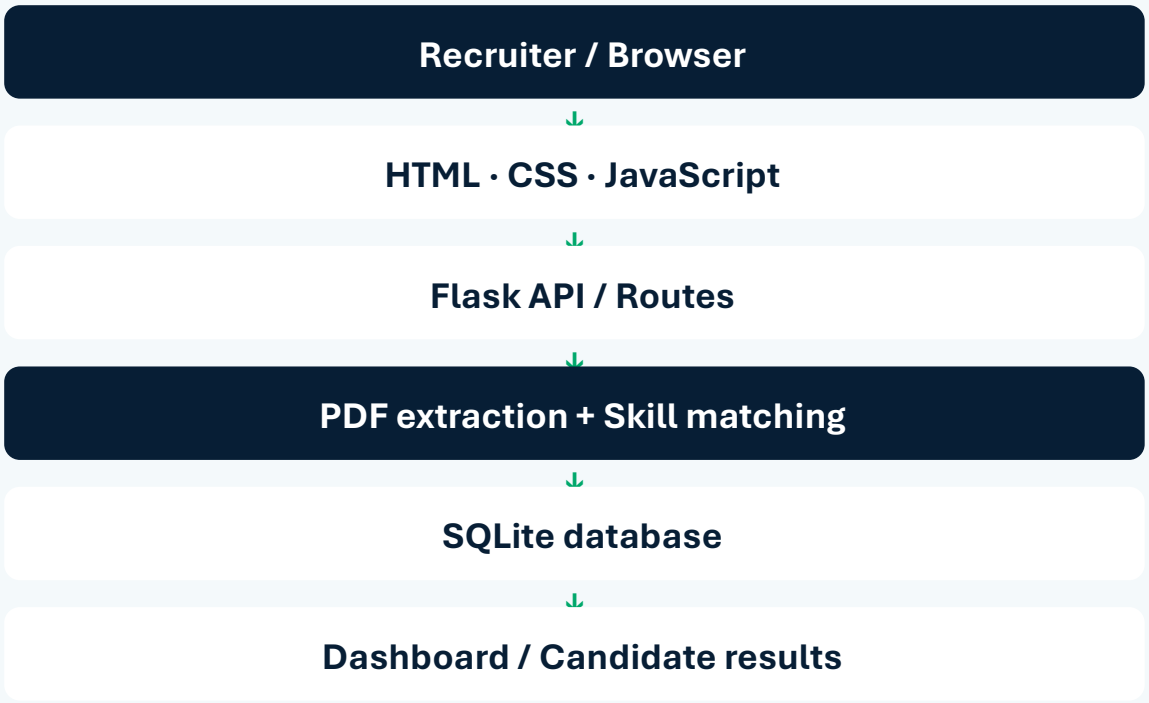
ADVANTAGE • Clear workflow

Central dashboard, candidate profiles and downloadable reports.

Technology stack

FRONTEND	HTML • CSS • JavaScript
BACKEND	Python • Flask
DOCUMENT PROCESSING	PDF text extraction
MATCHING ENGINE	Keyword / skill overlap scoring
DATA STORAGE	SQLite

System architecture



Implementation methodology

1 Input

Recruiter enters job details and uploads PDF files.

2 Parse

System extracts text from supported PDF resumes.

3 Identify

Matching routine searches for relevant skill terms.

4 Score

Calculate matched requirements as a percentage.

5 Present

Show ranked candidates with status indicators.

6 Export

Allow candidate review and CSV report download.

Frontend modules

Login & Dashboard

Landing page, overview cards and recent activity.

Resume Screening

Two-step job-details and PDF-upload flow.

Candidate Directory

Candidate list, match scores, search and view actions.

Candidate Profile

Overview, resume, skills, experience and education.

Reports & Settings

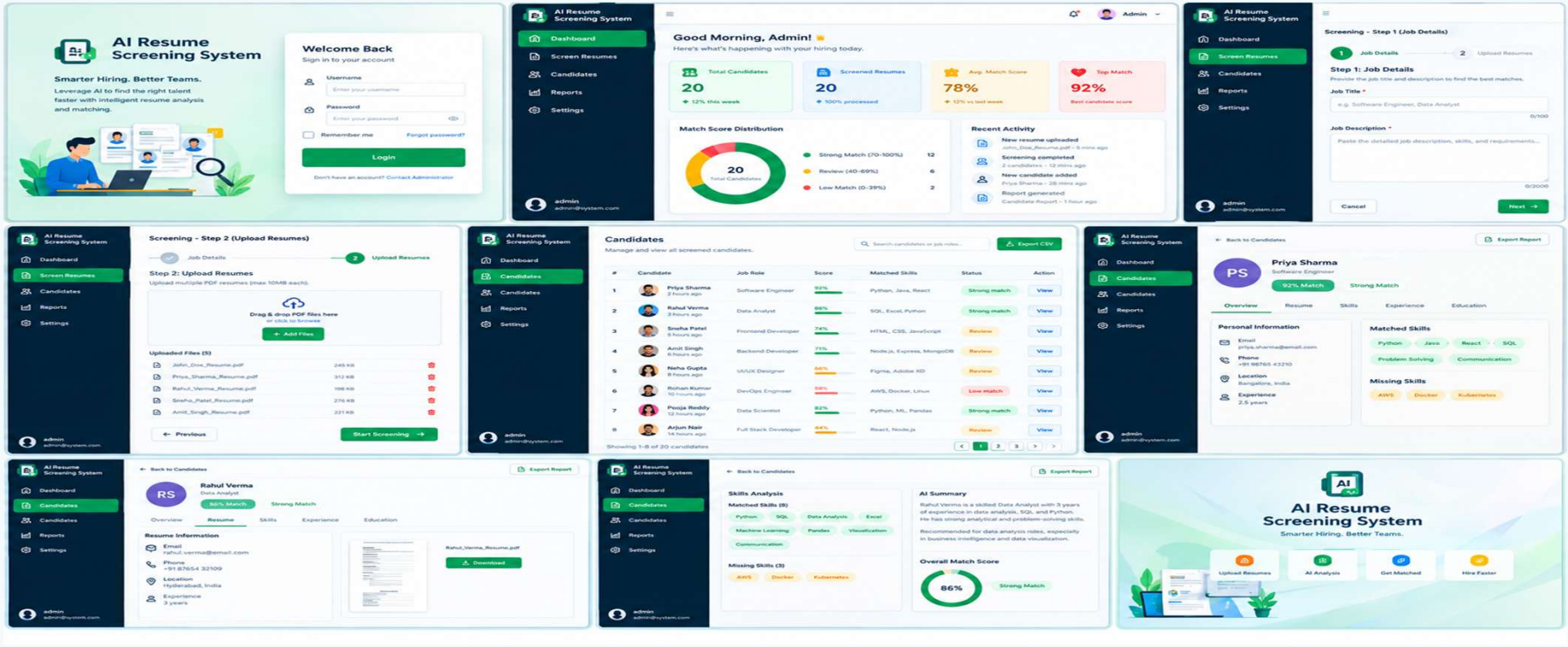
Summary views, export controls and preferences.

Navigation

Next / Previous controls and Dashboard access.

DESIGN REFERENCE

Interface preview



Reference interface: navy sidebar, green actions, white cards, resume screening and candidate views.

Backend & database

Flask web server

Serves the application and handles screening requests.

PDF text extraction

Reads available text from uploaded PDF resumes.

Matching logic

Computes keyword-based scores for candidate ranking.

SQLite storage

Persists screening and candidate information locally.

Testing & limitations

Workflow testing

Verify navigation, job entry, upload, screening and export.

Input validation

Check unsupported or empty documents and missing fields.

Matching limitations

Keyword matches may miss synonyms, context and experience depth.

Security limitations

Demo login is not production authentication; handle real resumes with care.

Future enhancements

Semantic matching

Use embeddings / NLP to understand related skills.

Authentication

Introduce secure accounts and role-based access.

File support

Add DOCX handling and improved PDF parsing.

Analytics

Advanced reporting, filters and evaluation metrics.

Conclusion

A practical prototype for faster initial resume shortlisting.

The system connects a modern web interface with Flask, PDF extraction, keyword matching and SQLite storage. It demonstrates an end-to-end screening workflow while leaving room for stronger AI matching, security and validation in future versions.

THANK YOU • QUESTIONS?

References

- Python and Flask official documentation
- HTML, CSS and JavaScript documentation (MDN)
- SQLite documentation
- Project source code and user interface reference
- PDF text extraction library documentation (see project requirements)

THANK YOU

AI RESUME SCREENING SYSTEM

Presented by: B.Chennakesavulu | III CST-A